

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D066.MD · PHASE 3 ●
LIVE

LECTURE · 量化金融 365

DAY 066 / 365

P3 · 数据与因子工程

DEEP DIVE · 8 页深度版

point-in-time 防未来函数

// PIT Data

KEY CONCEPTS · 三大要点

- ▶ 未来函数
- ▶ 披露日
- ▶ 版本数据

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

上一节我们用市值认清了时点和时段,这一节把不偷看未来这件事正式做成一套规范,叫 point-in-time,也就是时点数据规范。先说一个最容易踩、又最隐蔽的坑:未来函数。所谓未来函数,就是你在回测里,偷偷用了当时根本拿不到、要等到以后才知道的信息。财报就是重灾区。一份财报身上有两个日期:一个是报告期末,就是这个季度的最后一天,比如一季报对应3月31号;另一个是披露日,是这份财报真正对外公布、你才第一次能看到它的那天,通常要等到一两个月以后。很多人写回测图省事,在报告期末那天就把这季财报拿来用,可那天财报根本还没公布,等于偷看了一张还没贴出来的成绩单。这一节我们就用八只跨行业的真实股票,做一个买财报最好的票的简单策略,把偷看版和真实披露版放一起跑,亲眼看看前视偏差能把收益吹大多少。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

01 搞懂未来函数:回测里偷偷用了当时根本拿不到、要等以后才知道的信息,结论就会虚高

02 分清财报的两个日期:报告期末是这个季度的最后一天,披露日是财报真正公布、你才第一次能看到的那天,中间能隔一两个月

03 理解 point-in-time(时点数据规范):站在任何一天,只能用那天已经真正公布出来的数据,不能用以后才公布的

04 看清披露滞后造成的前视偏差:同一个买好财报的策略,按报告期末对齐(偷看)收益被吹大,按披露日对齐(真实)才是你真能拿到的

05 建立铁律:回测里用财报,必须用披露日对齐,绝不能用报告期末那天就当作已知

02 · CONTEXT · 历史与背景

小孙的财报策略回测时赚翻了,实盘却不灵,原来他在财报还没公布的那天就偷偷用上了财报

// 不读历史的策略走不远

小孙做了一个听起来很靠谱的策略:每个季度,挑出财报最好、利润涨得最猛的那批票买入。他回测了好几年,净值曲线漂亮得让他兴奋,涨幅远远跑赢大盘。他信心满满拿去实盘,结果跑了几个月,完全不是那么回事,收益平平甚至还亏。他百思不得其解,找一位做因子的前辈看代码。前辈翻了几眼就问:你这季财报,是按哪天算你知道了它?小孙说,按季度最后一天啊,一季报我就在3月31号用上。前辈摇头:坏就坏在这。一季报真正公布,得等到4月底甚至5月,你在3月31号就用,等于偷看了一张还没贴出来的成绩单。好财报的票,往往在财报公布那天会跳涨一下,你提前一个多月就埋伏进去,回测里自然把这段涨幅全捡了;可实盘里你压根不知道财报内容,这钱你赚不到。小孙这才明白,不是策略不行,是他的回测偷看了未来。这节课,我们就用八只真实股票,把小孙偷看的这段还原清楚,并教你用披露日对齐,彻底戒掉未来函数。

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

1. 未来函数:回测里偷用了当时拿不到的信息

未来函数,听着玄乎,其实一句话:你在回测的某一天,用了那天根本还没发生、或者还没公布、要等到以后才知道的信息。就好比穿越回上周一买彩票,却揣着这周的开奖号码,当然百发百中。回测里最常见的未来函数有三种:用了还没公布的财报、用了后来才修订的数据、用了今天才确定的股票名单去回测过去。它们的共同点都是站在未来回头操作,得出的好成绩全是假的,一到实盘就现原形。这一节我们专攻最高频的一种——还没公布的财报。

CONCEPT_02

2. 一份财报身上有两个日期:报告期末 vs 披露日

这是这节课的钥匙。任何一份财报,身上都挂着两个完全不同的日期。第一个叫报告期末,就是这份财报覆盖的那段时间的最后一天:一季报是3月31号,半年报是6月30号,年报是12月31号。第二个叫披露日,是这份财报真正编完、审完、对外公布,你这个普通投资者第一次能看到它的那天。关键是:这两个日期不是同一天,中间往往隔一两个月。一季报的内容,你得等到4月底5月初才看得到;年报更慢,常常要等到第2年34月。在披露日之前,这份财报对你来说是不存在的。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

3. point-in-time:站在那一天,只能用那天真公布了的数据

point-in-time,翻成人话就是时点数据规范,核心只有一句:你站在历史上的任何一天做决定,手里只能有那天真正已经公布出来的信息,一丁点未来的都不许碰。具体到财报,就是一份财报只能从它的披露日那天开始用,披露日之前你假装不知道它。做到这一点,你的回测才和真实的你站在那一天能做的决定完全一样。专业的数据供应商会专门卖一种叫 point-in-time 的数据库,贵就贵在它如实记录了每条数据是哪天才公布的,让你不会偷看。我们用免费的 baostock 也能做到——它的财报接口里就带着披露日这个字段。

CONCEPT_04

4. 披露滞后怎么就变成了前视偏差

把前面两点串起来,坑就出来了。假设你的策略是每季买财报最好的票。正确做法:等财报披露日到了,你看到了真实数据,再调仓。偷看做法:图省事,在报告期末那天就把这季财报拿来排序调仓——可那天财报还没公布。问题在于,好财报的票常常在财报公布前后会涨一波,这就是市场对好消息的反应。你按报告期末偷看,等于提前一个多月就埋伏进去,把这一波涨幅在回测里全捡了。于是偷看版的回测净值特别漂亮,但这部分收益是偷看未来偷来的,实盘里你根本拿不到。这就是披露滞后造成的前视偏差。

CONCEPT_05

5. PIT 是让回测等于实盘的关键

为什么要这么较真一个日期?因为回测的唯一价值,就是尽可能逼真地模拟如果当年我真这么做会怎样。一旦掺进未来函数,回测和实盘就成了两个世界:回测里你像开了天眼,实盘里你只是个凡人。两者的差距,就是你将来真金白银的亏损。point-in-time 这套规范,本质上是在强迫回测里的你,只能用当时凡人能拿到的信息,从而让回测的好成绩在实盘里也能复现。记住:一个不能在实盘复现的漂亮回测,比没有回测更危险,因为它会骗你重仓上场。

04 · PYTHON LAB · 真实可跑代码

财报有两个日期:报告期末(statDate)和披露日(pubDate)隔了一两个月;同一个买好财报策略,按报告期末对齐(偷看)vs 按披露日对齐(真实),回测净值差多少

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_066_point_in_time.py - PIT 防未来函数:财报披露滞后陷阱
# 真名上屏:baostock / query_growth_data → YOYNI(净利润同比) / pubDate(披露日) vs
statDate(报告期末) / 月末调仓 nlargest 多头
# 核心类比:财报"报告期末"是赛跑撞线那一刻,可成绩要等"披露日"裁判才公布;你在撞线那天就按成绩下注,
等于偷看了还没贴出来的成绩单 = 前视偏差(未来函数)
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import baostock as bs

def _data_path(_name):
    # 铁律62:data/ 放在 notebook 文件夹里。仓库根(run_lab)存取 out/notebook/data/;
    # 原版 notebook 在 out/notebook/ 用 data/;中国版在 out/notebook/cn/ 用 ../data/
    from pathlib import Path as _P
    _here = _P.cwd()
    for _b in [_here / 'data', _here / '..' / 'data', _here / 'out' / 'notebook' /
               'data', _here / '..' / '..' / 'data', _here / '..' / '..' / '..' / 'data']:
        if (_b / _name).exists():
            return str(_b / _name)
    if (_here / 'out' / 'notebook').exists():
        _t = _here / 'out' / 'notebook' / 'data'
    elif _here.name == 'cn':
```



```
_t = _here / '..' / 'data'  
else:  
_t = _here / 'data'
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/7)

财报有两个日期:报告期末(statDate)和披露日(pubDate)隔了一两个月;同一个买好财报策略,按报告期末对齐(偷看)vs 按披露日对齐(真实),回测净值差多少

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
_t.mkdir(parents=True, exist_ok=True)
return str(_t / _name)

pd.set_option('display.width', 160)
plt.rcParams['axes.unicode_minus'] = False

# ==== 标的池:8 只跨行业散户熟悉的票(财报波动大、披露滞后明显) ====
STOCKS = {
    '工商银行': 'sh.601398', '中国石油': 'sh.601857', '海螺水泥': 'sh.600585',
    '泸州老窖': 'sz.000568', '阳光电源': 'sz.300274', '歌尔股份': 'sz.002241',
    '保利发展': 'sh.600048', '温氏股份': 'sz.300498',
}
START, END = '2018-01-01', '2024-12-31'
CSV_PX = _data_path('D066_close.csv') # 前复权收盘(算策略真实收益)
CSV_FIN = _data_path('D066_earnings.csv') # 季度财报:statDate(报告期末)/pubDate(披露日)/YOYNI(净利润同比)

# ==== 0. 拉数据:前复权收盘 + 每季净利润同比增速及其披露日 ====
def _fetch():
    bs.login()
    px = {}
    fin_rows = []
    for name, code in STOCKS.items():
        rs = bs.query_history_k_data_plus(code, 'date,close', start_date=START,
                                          end_date=END, frequency='d', adjustflag='2')
        r = []
        while rs.error_code == '0' and rs.next():
```

```
r.append(rs.get_row_data())  
s = pd.DataFrame(r, columns=['date', 'close'])  
s['close'] = pd.to_numeric(s['close'], errors='coerce')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/7)

财报有两个日期:报告期末(statDate)和披露日(pubDate)隔了一两个月;同一个买好财报策略,按报告期末对齐(偷看)vs 按披露日对齐(真实),回测净值差多少

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
px[name] = s.set_index('date')['close']
for y in range(2017, 2025):
    for q in range(1, 5):
        rg = bs.query_growth_data(code=code, year=y, quarter=q)
        while rg.error_code == '0' and rg.next():
            g = rg.get_row_data() #
            code, pubDate, statDate, YOYEquity, YOYAsset, YOYNI, YOYEPSBasic, YOYPNI
            fin_rows.append({'name': name, 'pubDate': g[1], 'statDate': g[2], 'YOYNI': g[5]})
        bs.logout()
    return pd.DataFrame(px), pd.DataFrame(fin_rows)

if os.path.exists(CSV_PX) and os.path.exists(CSV_FIN):
    print(['数据] 从本地 CSV 读取 收盘 + 财报披露'])
    px = pd.read_csv(CSV_PX, parse_dates=['date']).set_index('date')
    fin = pd.read_csv(CSV_FIN)
else:
    print(['数据] baostock 拉取 8 只票 前复权收盘 + 季度净利润同比及披露日 ...'])
    px, fin = _fetch()
    px.index.name = 'date'
    px.to_csv(CSV_PX)
    fin.to_csv(CSV_FIN, index=False)
    print(['数据] 已存 CSV ->', CSV_PX)

px.index = pd.to_datetime(px.index)
fin['pubDate'] = pd.to_datetime(fin['pubDate'], errors='coerce')
fin['statDate'] = pd.to_datetime(fin['statDate'], errors='coerce')
fin['YOYNI'] = pd.to_numeric(fin['YOYNI'], errors='coerce')
fin = fin.dropna(subset=['pubDate', 'statDate', 'YOYNI'])

print('\n==== point-in-time:财报披露滞后造成的前视偏差 ====')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/7)

财报有两个日期:报告期末(statDate)和披露日(pubDate)隔了一两个月;同一个买好财报策略,按报告期末对齐(偷看)vs 按披露日对齐(真实),回测净值差多少

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('标的池 %d 只 · %s ~ %s' % (len(STOCKS), px.index[0].date(),
px.index[-1].date()))

# ==== 1. 披露滞后:报告期末 statDate 到真正能看到的 pubDate 隔了多少天 ====
fin['lag'] = (fin['pubDate'] - fin['statDate']).dt.days
lag_mean = float(fin['lag'].mean())
lag_max = int(fin['lag'].max())
print('\n[披露滞后] 报告期末 -> 披露日 平均滞后 %.0f 天,最长 %d 天' % (lag_mean,
lag_max))
print(fin.groupby(fin['statDate'].dt.to_period('Q'))
['lag'].mean().round(0).tail(8).to_string())

# ==== 2. 把同一份财报信号,挂到两个不同的"已知日期"上 ====
# peek(偷看): 信号在 报告期末 statDate 当天就当作已知 — 其实那天财报还没公布
# pit (真实): 信号在 披露日 pubDate 当天才当作已知 — 这才是当时真能拿到的
monthly = px.resample('ME').last() # 月末收盘
mret = monthly.pct_change().shift(-1) # 本月末建仓 -> 下月末的收益

def signal_panel(date_col):
    panel = {}
    for name in STOCKS:
        sub = fin[fin['name'] == name].sort_values(date_col)
        ser = pd.Series(sub['YOYNI'].values, index=sub[date_col])
        ser = ser[~ser.index.duplicated(keep='last')].sort_index()
        panel[name] = ser.reindex(monthly.index, method='ffill')
    return pd.DataFrame(panel)

sig_peek = signal_panel('statDate')
```

```
sig_pit = signal_panel('pubDate')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/7)

财报有两个日期:报告期末(statDate)和披露日(pubDate)隔了一两个月;同一个买好财报策略,按报告期末对齐(偷看)vs 按披露日对齐(真实),回测净值差多少

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
def backtest(sig):
    half = max(1, len(STOCKS) // 2)
    eq, dates = [1.0], [monthly.index[0]]
    for i in range(len(monthly.index) - 1):
        t = monthly.index[i]
        s = sig.loc[t].dropna()
        r_next = mret.loc[t]
        if len(s) >= half:
            longs = s.nlargest(half).index # 买"财报最好"的一半,等权
            r = r_next[longs].mean()
        else:
            r = r_next.mean() # 信号不足时持有全部(基准)
        eq.append(eq[-1] * (1 + (0.0 if pd.isna(r) else r)))
        dates.append(monthly.index[i + 1])
    return pd.Series(eq, index=dates)

eq_peek = backtest(sig_peek)
eq_pit = backtest(sig_pit)
ret_peek = (eq_peek.iloc[-1] - 1) * 100
ret_pit = (eq_pit.iloc[-1] - 1) * 100
gap = ret_peek - ret_pit
print('\n[同一个"买好财报"策略,只换信号的已知日期]')
print('偷看(报告期末就用) 总收益 %.1f%% 终值 %.3f' % (ret_peek, eq_peek.iloc[-1]))
print('真实(披露日才用) 总收益 %.1f%% 终值 %.3f' % (ret_pit, eq_pit.iloc[-1]))
print('前视偏差 = %.1f 个百分点(同一策略、同一批票,只因偷看了还没公布的财报)' % gap)

# ==== 3. 单例放大:哪一季,偷看者在"报告期末->披露日"之间白捡了一段涨幅 ====
hot = fin[fin['YOYNI'] > 0].copy()
moves = []
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/7)

财报有两个日期:报告期末(statDate)和披露日(pubDate)隔了一两个月;同一个买好财报策略,按报告期末对齐(偷看)vs 按披露日对齐(真实),回测净值差多少

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
for _, row in hot.iterrows():
    win = px[(px.index >= row['statDate']) & (px.index <= row['pubDate'])]
    if len(win) >= 5 and row['name'] in px.columns:
        seg = win[row['name']].dropna()
        if len(seg) >= 5:
            moves.append((row['name'], row['statDate'], row['pubDate'], seg.iloc[-1] /
                           seg.iloc[0] - 1, row['YOYNI']))
            mv = pd.DataFrame(moves, columns=['name', 'statDate', 'pubDate', 'segret',
                                               'yoyni']).sort_values('segret', ascending=False)
            top = mv.iloc[0]
            print('\n[单例] %s 某季净利同比 %.0f%%, 报告期末 %s -> 披露日 %s 之间股价已走 %.1f%%' %
                  (
                    top['name'], top['yoyni'] * 100, top['statDate'].date(), top['pubDate'].date(),
                    top['segret'] * 100))
            print(' 偷看者在财报还没公布时就提前下注,把这段涨幅白捡了 —— 这正是前视偏差的来源')

# ==== 4. 三张图 ====
# 图1:披露滞后分布(报告期末 -> 披露日 隔多少天)
fig, ax = plt.subplots(figsize=(10, 6))
ax.hist(fin['lag'], bins=24, color='#1565c0', alpha=.85)
ax.axvline(lag_mean, ls='--', c='red', lw=2)
ax.text(lag_mean + 2, ax.get_ylim()[1] * 0.9, '平均滞后 %.0f 天' % lag_mean,
        color='red', fontweight='bold')
ax.set_title('财报"报告期末"到"披露日"平均隔 %.0f 天:这段时间你其实还看不到这份财报' %
             lag_mean)
ax.set_xlabel('报告期末 -> 披露日的滞后天数')
ax.set_ylabel('财报次数')
ax.grid(alpha=.3, axis='y')
fig.tight_layout()
fig.savefig('chart_01.png', dpi=110)
plt.close()
```

图2:两条净值曲线 —— 偷看 vs 真实


```
fig, ax = plt.subplots(figsize=(11, 6))
ax.plot(eq_peek.index, eq_peek.values, lw=2, c='#c62828', label='偷看:报告期末就用财报 (前视)')
ax.plot(eq_pit.index, eq_pit.values, lw=2, c='#2e7d32', label='真实:披露日才用财报 (PIT)')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (7/7)

财报有两个日期:报告期末(statDate)和披露日(pubDate)隔了一两个月;同一个买好财报策略,按报告期末对齐(偷看)vs 按披露日对齐(真实),回测净值差多少

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ax.axhline(1.0, c='k', lw=.8)
ax.text(eq_peek.index[-1], eq_peek.iloc[-1], ' %+.0f%%' % ret_peek,
color='#c62828', fontweight='bold', va='center')
ax.text(eq_pit.index[-1], eq_pit.iloc[-1], ' %+.0f%%' % ret_pit, color='#2e7d32',
fontweight='bold', va='center')
ax.set_title('同一个"买好财报"策略:偷看版 vs 真实披露版,终值差 % .0f 个百分点(全是前视偏差吹出
来的)' % gap)
ax.set_ylabel('净值(起点=1)')
ax.legend(loc='upper left')
ax.grid(alpha=.3)
fig.tight_layout()
fig.savefig('chart_02.png', dpi=110)
plt.close()
```

图3:单例放大 — 报告期末到披露日之间,偷看者白捡的那段涨幅

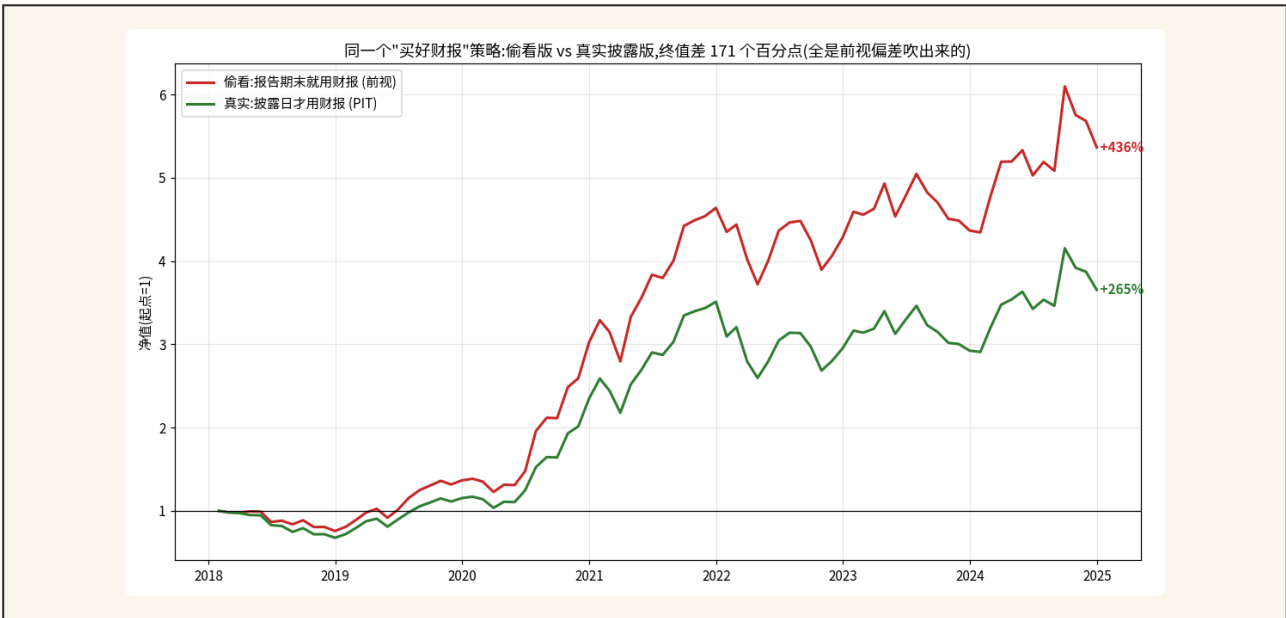
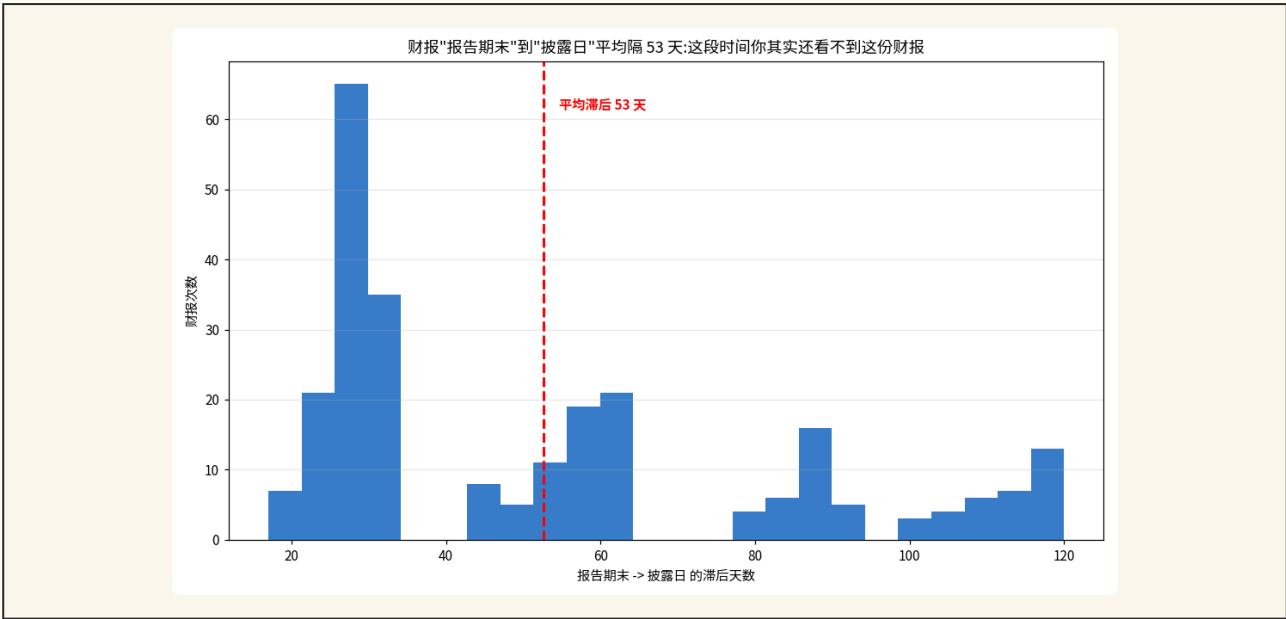
```
fig, ax = plt.subplots(figsize=(11, 6))
seg_lo = top['statDate'] - pd.Timedelta(days=20)
seg_hi = top['pubDate'] + pd.Timedelta(days=20)
seg = px[(px.index >= seg_lo) & (px.index <= seg_hi)][top['name']].dropna()
ax.plot(seg.index, seg.values, lw=1.8, c='#333')
ax.axvline(top['statDate'], ls='--', c='#c62828', lw=2)
ax.axvline(top['pubDate'], ls='--', c='#2e7d32', lw=2)
ax.axvspan(top['statDate'], top['pubDate'], color='#ffcc80', alpha=.5)
ax.text(top['statDate'], seg.max(), ' 报告期末(财报还没公布)', color='#c62828',
fontweight='bold', va='top')
ax.text(top['pubDate'], seg.min(), ' 披露日(这天才真能看到)', color='#2e7d32',
fontweight='bold', ha='right', va='bottom')
ax.set_title('%s:报告期末到披露日之间股价已走 %+.0f%%,偷看者把这段(橙色区)白捡了' %
(top['name'], top['segret'] * 100))
ax.set_ylabel('前复权收盘价(元)')
ax.grid(alpha=.3)
fig.tight_layout()
fig.savefig('chart_03.png', dpi=110)
```

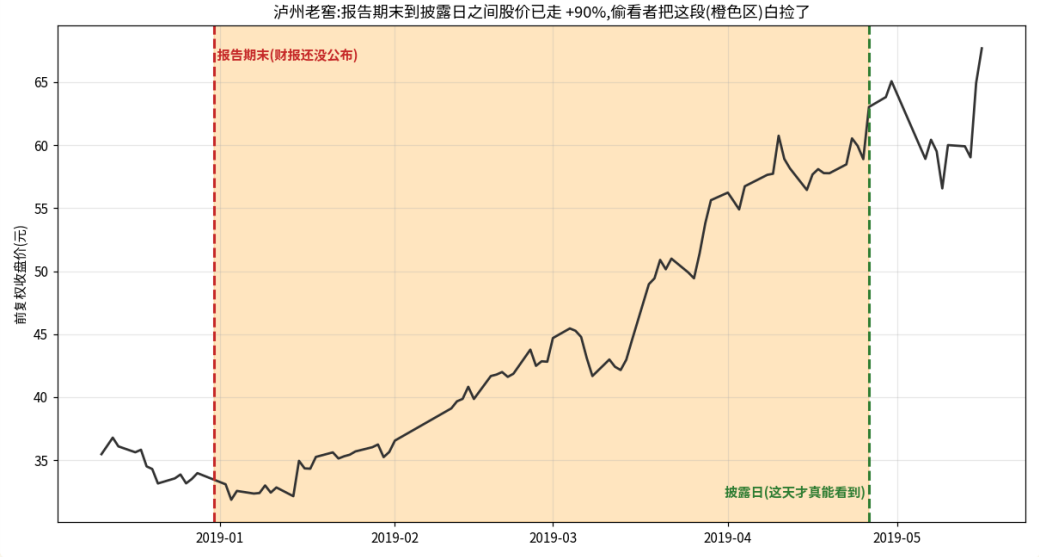
```
plt.close()
```

```
print('\n[done] PIT 披露滞后前视偏差实证完成 — 3 图已出')
```

回测结果

// · matplotlib 真实输出





偷看版总收益(报告期末就用财报)

+436.4%(终值5.36,前视)

真实版总收益(披露日才用财报)

+265.4%(终值3.65,PIT)

前视偏差

约 +171 个百分点(凭空多出来的假收益)

财报平均披露滞后

53 天(年报最久约101天,最长120天)

泸州老窖单例

报告期末→披露日之间股价已涨 +90.5%

标的与区间

8 只跨行业票,2018-2024 月度调仓

▶ 01 财报有两个日期,中间隔一两个月(图1)

图1把8只票所有季度财报的披露滞后画成直方图:从报告期末到真正披露,平均隔53天,最长能到120天。换句话说,一份财报在它覆盖的那个季度结束时根本还没出来,你得再等一两个月才看得到。红色竖线标出平均53天的位置,大部分财报都堆在30到100天这一段。

▶ 02 偷看版净值全程压着真实版(图2)

图2两条净值曲线,红线是偷看版(报告期末当天就用这季财报),绿线是真实版(等到披露日才用)。红线从头到尾都压在绿线上方,因为好财报的票往往在财报公布前后就开始涨,偷看者提前一个多月埋伏进去,把这段涨幅全捡了。一样的策略、一样的票,只因为对齐的日期不同,红线终值5.36、绿线只有3.65。

▶ 03 前视偏差凭空吹出171个点

偷看版总收益+436%,真实版+265%,差了整整171个百分点。这171个点全是前视偏差吹出来的假收益:它来自你偷看了还没公布的财报,实盘里你压根拿不到。很多人回测时净值漂亮得不行,一到实盘就打回原形,差的往往就是这种偷看出来的水分。

▶ 04 单例:泸州老窖滞后期股价已涨九成(图3)

图3放大一个真实例子:泸州老窖2018年的年报,净利润同比涨了35%,报告期末是2018年12月31号,可披露日要等到2019年4月26号。在这中间一百多天里,它的股价已经从约33元涨到约63元,涨了90%。橙色阴影就是偷看者白捡的这段——财报还没公布,他靠偷看提前埋伏,实盘里你根本不可能在年报出来前就知道它这么好。

▶ 05 越是年报,滞后越久,越要小心

按报告类型看滞后:一季报大约27天,半年报约56天,三季报约28天,而年报最久,要等约101天。也就是说去年的年报,你常常要等到今年的4月才看得到。回测里如果你在12月31号就用上整年的年报数据,等于偷看了34个月。年报是滞后的重灾区,做因子时尤其要用披露日对齐。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

前视偏差·买
好财报策略

N/A

同一个『每季买财报最好的票』策略,8只跨行业票
2018-2024月度调仓:按报告期末对齐(偷看,财报还没公布)
总收益+436.4%,按披露日对齐(真实,当时真能拿到)
+265.4%,前视偏差凭空+171个百分点。

披露滞后·泸
州老窖年报

sz.000568

泸州老窖2018年报净利润同比+35%,报告期末2018年12月31号但披露日要等到2019年4月26号,中间约116天股价已从约33元涨到约63元(+90.5%),偷看者提前埋伏把这段涨幅白捡,实盘根本拿不到。

年报滞后最
久·全样本

N/A

8只票实测,按报告类型看披露滞后:一季报约27天、半年报约56天、三季报约28天、年报最久约101天,最长120天。年报是滞后重灾区,12月31号就用整年年报等于偷看34个月。

偷看净值全
程领先

N/A

偷看版净值曲线(终值5.36)从头到尾压着真实版(终值3.65),因为好财报票在披露前后跳涨、偷看者提前一个多月埋伏;这段超额在实盘里完全复现不了,回测漂亮实盘打回原形差的就是这种水分。

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

用报告期末日期对齐财报,等于偷看还没公布的财报

财报在报告期末那天根本还没公布,你那天就拿来用,就是经典未来函数。必须用披露日对齐。

△ 02

以为季度一结束财报就能用,忽略一两个月的披露滞后

一季报要等4月底5月,年报常等到第2年34月。在披露日之前,这份财报对你不存在。

△ 03

用后来修订过的最终财报值回测,而不是首次披露的原始值

财报可能事后更正、重述,你拿最终修订值回测又是偷看未来,要用当时首次披露的版本快照。

△ 04

用今天的股票池/指数成分名单去回测历史,埋进幸存者偏差

今天的名单里全是活下来的赢家,用它回测过去等于又偷看了未来,是未来函数的近亲。

△ 05

把披露滞后当成一个固定天数

季报、年报滞后差别很大,不同公司也不同。别拍脑袋统一减30天,要用每份财报真实的披露日。

point-in-time — 几条铁规矩

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 回测里用任何财报,一律用披露日对齐,披露日之前假装这份财报不存在。
- 02 心里永远记着财报有两个日期:报告期末是这季最后一天,披露日才是你能看到它的那天。
- 03 用首次披露的原始数据,别用后来修订过的最终值,那也是偷看未来。
- 04 股票池、指数成分要用历史上当时的真实名单,别用今天的名单回测过去。
- 05 回测出好成绩前先自问:这条信息,在那一天我这个凡人真的已经能看到了吗?

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

07 · RECAP · 一页课件总结

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 未来函数=回测里偷用了当时根本拿不到、要等以后才知道的信息,结论会虚高,一到实盘就现原形。
- 02 一份财报有两个日期:报告期末是这季最后一天,披露日是财报真正公布、你才第一次看得那天,中间隔一两个月。
- 03 point-in-time(时点数据规范):站在历史任意一天,只能用那天已经真公布出来的数据;财报只能从披露日起用。
- 04 披露滞后变前视偏差:在报告期末就用财报=偷看,好财报票披露前后跳涨,偷看者提前埋伏把涨幅白捡。
- 05 本课实测:同一个买好财报策略,偷看版+436.4% vs 真实版+265.4%,前视偏差凭空+171个点;平均滞后53天、年报约101天。
- 06 铁律:回测里用财报一律按披露日对齐,披露日之前假装这份财报不存在;还要警惕数据修订和幸存者偏差两种近亲。

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 一份财报身上有哪两个日期?它们各是什么意思?回测里用财报,应该按哪个日期对齐才不算偷看未来?

Q02 什么叫未来函数?用报告期末那天就拿这季财报来排序选股,为什么是一种未来函数?会让回测结论怎样?

Q03 point-in-time(时点数据规范)的核心要求是什么?除了财报披露滞后,还有哪两种常见的未来函数?

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Marcos López de Prado 《Advances in Financial Machine Learning》(2018/Wiley)– 第一部分专讲回测里的各种偏差,未来函数和 point-in-time 是核心,实战派必读。
- ▶ Campbell Harvey, Yan Liu 《Backtesting》(2015/SSRN)– 系统讨论回测可信度,未来函数和数据窥探如何制造虚假的好成绩。
- ▶ Marcos López de Prado 《The 7 Reasons Most Machine Learning Funds Fail》(2018/SSRN)– 七大翻车原因里,数据泄露和未来函数排在最前面,短小精悍。
- ▶ David Bailey, Marcos López de Prado 《The Probability of Backtest Overfitting》(2014/SSRN)– 讲漂亮回测如何骗人,和未来函数同根同源。
- ▶ baostock 官方文档 query_growth_data / query_profit_data – 看清每个财报字段都带 pubDate(披露日)和 statDate(报告期末),这是动手做 PIT 的第一步。

NEXT LECTURE · 下一节预告

DAY 067 因子的定义和分类

这一节我们用披露日对齐,堵住了财报这个最大的未来函数漏洞。从下一节开始,我们正式进入因子的世界。下一节先把因子是什么、有哪些大类讲清楚:因子,简单说就是给所有股票打分的一把尺子;价值、成长、质量、规模、动量,就是几把不同的尺子。我们会用真实数据给一篮子股票用五把尺子各打一遍分,你会亲眼看到,同一只票用不同尺子量,排名能天差地别。